NATIONAL ECONOMICS UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

CHAPTER 4

Building models and developing AI applications

The workflow from data to application · Application of AI in Business and Management

MSc. Nguyen Quoc Thai

quocthai@neu.edu.vn

Course content

- Chapter 1: Overview of Artificial Intelligence and supporting tools
- Chapter 2: Problem solving of Artificial Intelligence and data science
- Chapter 3: Machine learning and deep learning methods
- **Chapter 4: Building models and developing AI applications in business and management**
  - **The workflow, step by step, and the group project on Kaggle (this lecture)**

Objectives

After this chapter you can

objective	where in this chapter
Name the eight steps of building an AI application and the main task of each	Section 01
Carry out each step with pandas, matplotlib/seaborn and scikit-learn	Section 02, a small table
Apply the workflow to real data and make a first Kaggle submission	Section 03, Card Churn
Build a simple interface that uses a trained model for a new customer	Step 8 and Section 03

**One workflow, two datasets: a small table to learn each step,
then Card Churn, the first competition of the group project.**

Outline

One workflow, learnt on a small table, then applied to Kaggle data

01

The workflow

eight steps

- the steps at a glance
- keywords and tools

02

Step by step

306 rows

- short code and output
- Practice 1 and 2

03

Card Churn

Kaggle data

- the same eight steps
- first submission

04

Summary

group project

- four rules
- Cafe Revenue, project

Session 1: Section 01 and Steps 1–6
Session 2: Steps 7–8, Section 03, Section 04 and the quiz

01

The workflow

Eight steps from a CSV file to an application, and the keywords of each step.



From a CSV file to an AI application

Chapter 3 trained one model at a time. Chapter 4 connects the steps from data to an application.



Data

customers_toy.csv

306 rows, 9 columns



Model

LogisticRegression

learns P(churn) from the data



Interface

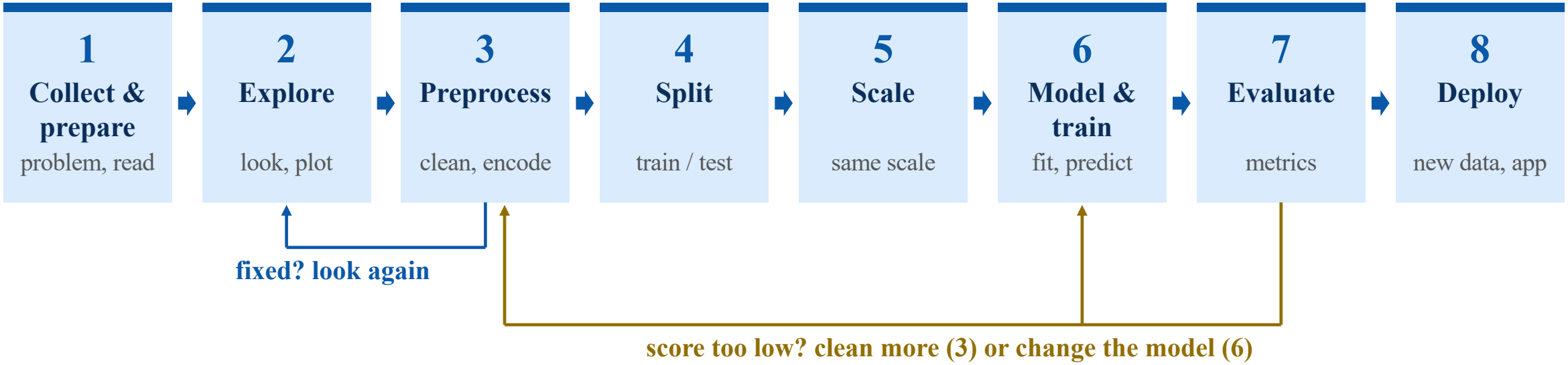
a form on Colab

"Churn risk: 0.94 → call"

A bank officer does not open a notebook. She types a customer's details into a form and reads one number: the risk that this customer cancels the card.
Every step before that is there to make this number trustworthy.

The workflow at a glance

Eight steps, with two ways back



Steps 1–3 work on the data · Steps 4–7 on the model · Step 8 on the user

The workflow in detail

Keywords and main tools of each step

step	keywords	main tools
1 Collect & prepare	problem type, target, metric, read, merge, data dictionary	pd.read_csv, merge
2 Explore	shape, types, statistics, missing values, counts, target balance, charts	pandas, matplotlib, seaborn
3 Preprocess	duplicates, missing and wrong values, spelling, types, encoding	pandas
4 Split	train / test, stratify, random_state (k-fold, time split)	train_test_split
5 Scale	min-max, standard, fit on train only	MinMaxScaler, StandardScaler
6 Model & train	model choice, baseline, fit, hyperparameters	scikit-learn models
7 Evaluate	predict, confusion matrix, accuracy, F1, MAE, MSE, overfitting	sklearn.metrics
8 Deploy	new data, predict, interface, export a file	Colab form, to_csv

02

Step by step

Each step on a small table of 306 customers: the usual tasks, short code and its output.



The dataset for Section 02

customers_toy.csv · 306 rows of a bank's card customers

column	meaning	values
customer_id	customer code	C001 … C300
bank	bank name	one value
age	age	years
gender	gender	M, F
city	city	Ha Noi, Ho Chi Minh, Da Nang
income	monthly income	million VND
txn_3m	transactions in 3 months	count
open_year	year the card was opened	2018–2025
churn	target: cancelled the card	0 / 1

Why a small table

- 306 rows: every output fits on a slide
- A few errors are planted on purpose
- Step 2 finds them, Step 3 fixes them
- Notebook: 4.1_workflow_toy.ipynb

Step 1 · Collect & prepare

What to do in this step

Usual tasks

- Define the problem: what to predict, which type, which metric
- Get the data and read it into a DataFrame
- Merge tables that share a key column
- Write a data dictionary: name, meaning, unit, type

Know the names

- CSV and Excel files
- Databases (SQL)
- APIs
- Kaggle datasets

Output of this step: one table, and a clear question to answer with it.

Define the problem first

Step 1 · Collect & prepare

business question	problem type	target	metric
Will this customer cancel the card?	classification	churn: 0 / 1	accuracy, F1
How much will the shop sell today?	regression	revenue	MAE, MSE
Which groups of customers do we have?	clustering	no target	business sense

Three questions before any code:
What do we predict? From which columns? Measured by what?
The answers decide the model (Step 6) and the metric (Step 7).

Read and combine tables

Step 1 · Collect & prepare


```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("data/customers_toy.csv")    # one row per customer
df.shape                                       # (rows, columns)
```

```
(306, 9)
```

```
branches = pd.read_csv("data/branches.csv")  # one row per city
merged = df.merge(branches, on="city", how="left")
merged[["customer_id", "city", "region", "n_branches"]].head(3)
```

```
  customer_id         city region  n_branches
0        C179  Ho Chi Minh  South          15
1        C268       Ha Noi  North          12
2        C192       Ha Noi  North          12
```

Where the file is

- own computer: next to the notebook
- Colab: upload, or mount Google Drive
- Kaggle: under /kaggle/input/

merge

- on: the shared column, here city
- how="left": keep every row on the left

Compare shape before and after a merge: more rows means the key is repeated in the right table.

Step 2 · Explore

What to do in this step

Usual tasks

- Size, column types, summary statistics
- Missing values in each column
- Categories of each text column
- Balance of the target classes
- Charts: one column, two columns, correlation

Know the names

- pairplot
- outliers: the 1.5 × IQR rule of the boxplot

Output of this step: a written list of problems, before any of them is fixed.

First look: info() and describe()

Step 2 · Explore


```
1 df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 306 entries, 0 to 305
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   customer_id  306 non-null    str
 1   bank         306 non-null    str
 2   age          306 non-null    int64
 3   gender       306 non-null    str
 4   city         306 non-null    str
 5   income       291 non-null    float64
 6   txn_3m       306 non-null    int64
 7   open_year    306 non-null    int64
 8   churn        306 non-null    int64
dtypes: float64(1), int64(4), str(4)
memory usage: 21.6 KB
```

```
1 df.describe().round(1)
```

	age	income	txn_3m	open_year	churn
count	306.0	291.0	306.0	306.0	306.0
mean	42.9	19.8	19.5	2021.5	0.3
std	56.2	8.9	6.7	2.3	0.5
min	0.0	6.0	5.0	2018.0	0.0
25%	32.0	14.1	15.0	2020.0	0.0
50%	39.5	18.0	19.0	2022.0	0.0
75%	46.0	23.8	23.8	2023.0	1.0
max	999.0	75.3	47.0	2025.0	1.0

Look for

- income: 291 non-null of 306, so 15 empty cells
- age: min 0 and max 999 cannot be true; std 56.2 is large because of them
- str = text column: describe() skips it (next slide: value_counts)

Missing values, categories, target

Step 2 · Explore


```
1 df.isna().sum()
```

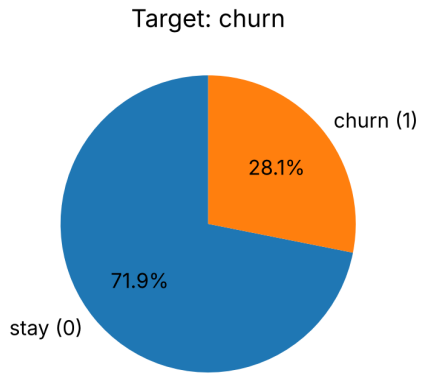
```
customer_id      0
bank             0
age              0
gender           0
city             0
income          15
txn_3m           0
open_year        0
churn            0
```

```
1 df.gender.value_counts(dropna=False)
```

```
gender
M       144
F       130
Nam      10
Nữ        8
 F        7
male      7
```

```
1 df.churn.value_counts(normalize=True).round(3)
```

```
churn
0    0.719
1    0.281
```

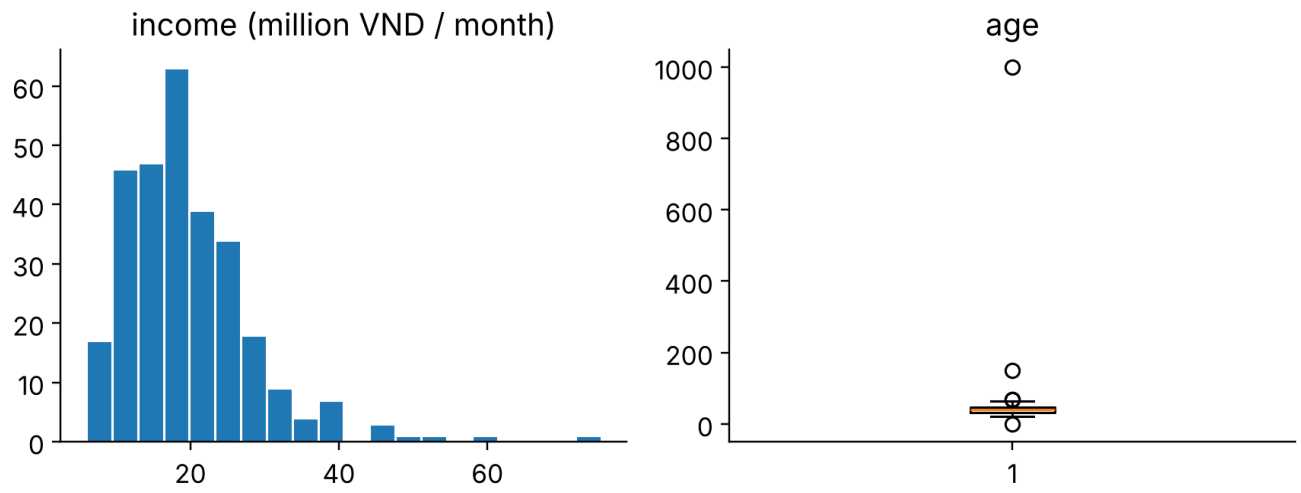


income: 15 missing · gender: six spellings of two values ("Nam", "male" = M; "Nữ", " F" = F)
28.1% churn: "nobody churns" is right 71.9% of the time, the baseline idea of Step 6

One column: histogram and boxplot

Step 2 · Explore


```
fig, ax = plt.subplots(1, 2, figsize=(8, 3.2))
ax[0].hist(df.income.dropna(), bins=20, color="#1f77b4", edgecolor="white")
ax[0].set_title("income (million VND / month)")
ax[1].boxplot(df.age.dropna())            # dropna: boxplot cannot draw NaN
ax[1].set_title("age")
plt.tight_layout()
plt.show()
```

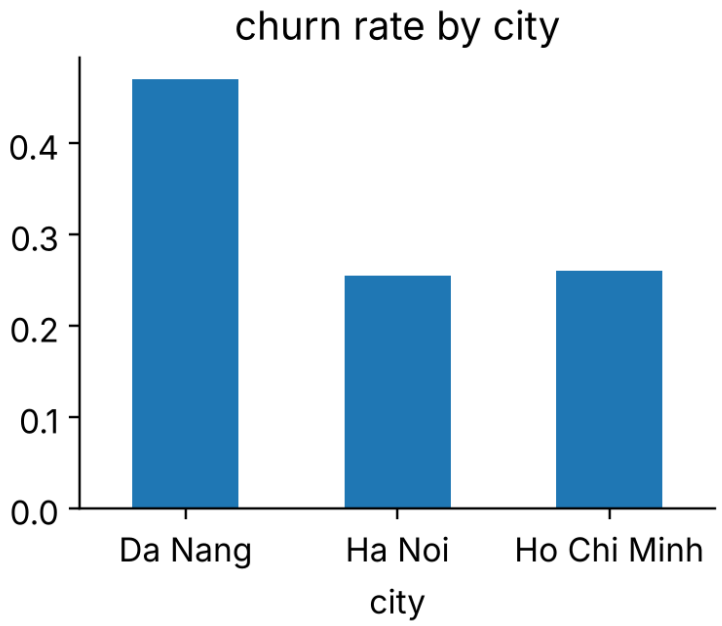


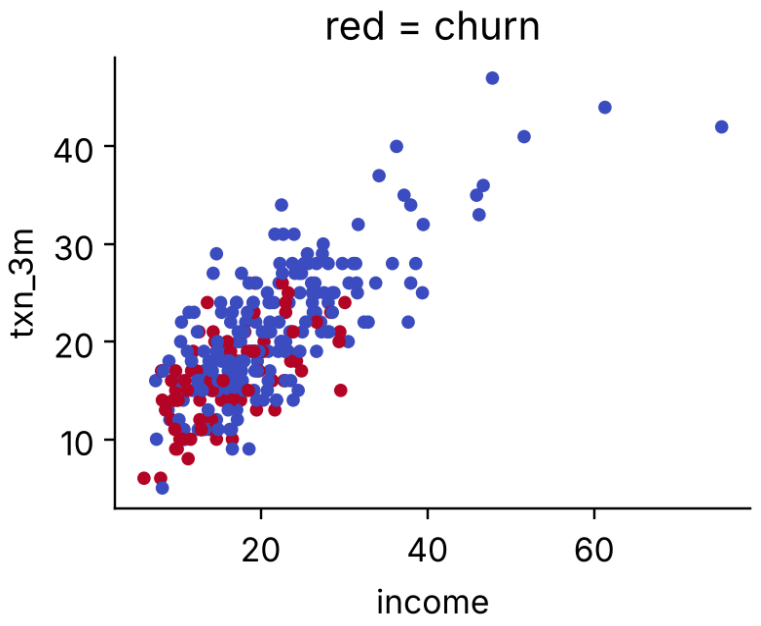
Read the charts

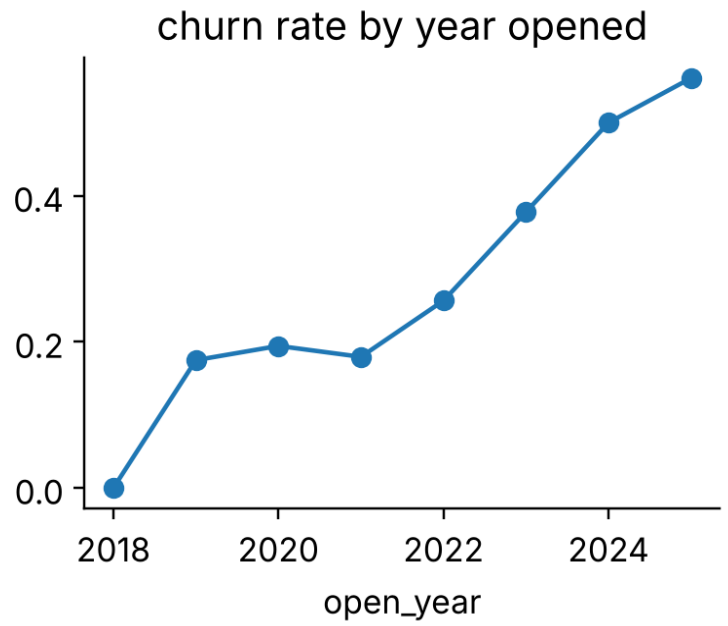
- Histogram: income is skewed right, most below 30 million
- Boxplot: 999 and 150 sit far above the rest
- dropna(): a boxplot draws nothing if the column has NaN

Two columns: bar, scatter, line

Step 2 · Explore







Bar · compare groups

`df.groupby("city").churn.mean()`

Da Nang churns most (47%)

Scatter · two numbers

`ax[1].scatter(df.income, df.txn_3m, c=...)`

churners (red) have few transactions

Line · an ordered axis

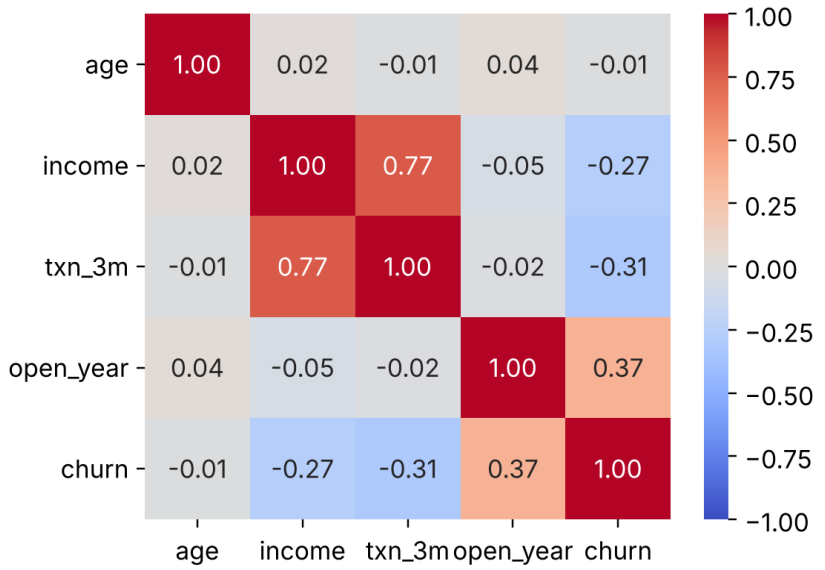
`df.groupby("open_year").churn.mean()`

newer cards churn more

Correlation: heatmap

Step 2 · Explore


```
corr = df.corr(numeric_only=True)
plt.figure(figsize=(5.2, 4))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.show()
```



Read the row churn

- txn_3m −0.31, income −0.27: fewer transactions, lower income → more churn
- open_year +0.37: newer cards churn more
- age −0.01, but only because of 0, 150, 999: after Step 3 it is 0.12
- Correlation is not causation

Which chart for which question

Step 2 · Explore

question	chart	call
How is one numeric column spread?	histogram	`plt.hist`
Are there values far from the rest?	boxplot	`plt.boxplot`
How do groups differ?	bar	`groupby(...).mean().plot(kind="bar")`
How are two numeric columns related?	scatter	`plt.scatter`
How does a value change over time?	line	`plot(marker="o")`
What share does each part have?	pie	`plt.pie`
How are many numeric columns related?	heatmap, pairplot	`sns.heatmap, sns.pairplot`

Step 3 · Preprocess

What to do in this step

Usual tasks

- Drop duplicate rows
- Drop constant columns and ID columns
- Turn wrong values into missing values
- Unify different spellings of one value
- Handle missing values
- Encode text columns as numbers

Know the names

- outliers
- new features
- class imbalance
- feature selection, PCA

Output of this step: a table of numbers only, with no duplicates and no missing values.

Duplicates and useless columns

Step 3 · Preprocess


```
print("duplicates:", df.duplicated().sum())
df = df.drop_duplicates()
df.shape
```

```
duplicates: 6
(300, 9)
```

```
df = df.drop(columns=["customer_id", "bank"])
```

```
df.nunique()
```

```
customer_id    300
bank             1
age             48
gender           6
city             3
income         185
txn_3m          38
open_year        8
churn            2
```

bank has 1 value, customer_id has 300: neither helps a model.
6 duplicate rows dropped: 306 → 300.

Wrong values and inconsistent text

Step 3 · Preprocess


```
df.loc[~df.age.between(18, 100), "age"] = np.nan    # 0, 150, 999 -> NaN
df["gender"] = (df.gender.str.strip()                 # " F" -> "F"
                  .replace({"Nam": "M", "male": "M", "Nữ": "F"}))
df.gender.value_counts()
```

```
gender
M    159
F    141
```

before (306)	count
M	144
F	130
Nam	10
Nữ	8
" F"	7
male	7

after (300)	count
M	159
F	141

A wrong value becomes NaN; the row stays, because its other columns are still useful.
str.strip() removes spaces at both ends; replace() maps each spelling to one value.

Missing values

Step 3 · Preprocess


```
print(df[["age", "income"]].isna().sum())            # before
fill = df[["age", "income"]].median()
df = df.fillna(fill)
print("missing after:", df.isna().sum().sum())
```

```
age         3
income     15
missing after: 0
```

way	call	when
drop the rows	`dropna()`	few rows have gaps
fill: median (numbers)	`fillna(...)`	the default
fill: mode (text)	`fillna(...)`	the default

Strictly, the median should be learnt from the train set only (Step 4), or the test set leaks into training. On this small table the effect is tiny; Section 03 does it properly with SimpleImputer inside a pipeline.

Encoding: text columns to numbers

Step 3 · Preprocess


```
1 df = pd.get_dummies(df, columns=["gender", "city"], dtype=int)
2 df.columns.tolist()
```

```
['age',
 'income',
 'txn_3m',
 'open_year',
 'churn',
 'gender_F',
 'gender_M',
 'city_Da Nang',
 'city_Ha Noi',
 'city_Ho Chi Minh']
```

city	city_Da Nang	city_Ha Noi	city_Ho Chi Minh
Ha Noi	0	1	0
Da Nang	1	0	0
Ho Chi Minh	0	0	1

One-hot encoding

- one 0/1 column per value
- gender → gender_F, gender_M
- 10 columns, all numbers

Step 4 · Split

What to do in this step

Usual tasks

- Separate the target y from the features X
- Keep 20% of the rows as the test set
- Keep the class shares with stratify=y
- Fix the split with random_state

Know the names

- validation set
- k-fold cross-validation
- time-based split

Output of this step: a train set to learn and decide on, and a test set kept aside until Step 7.

Train and test

Step 4 · Split

train: 80% · 240 rows · learn and decide | **test: 20% · 60 rows**

```
from sklearn.model_selection import train_test_split

X = df.drop(columns="churn")      # features
y = df.churn                      # target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
print(X_train.shape, X_test.shape)
print(round(y_train.mean(), 3), round(y_test.mean(), 3))
```

```
(240, 9) (60, 9)
0.288 0.283
```

Read the output

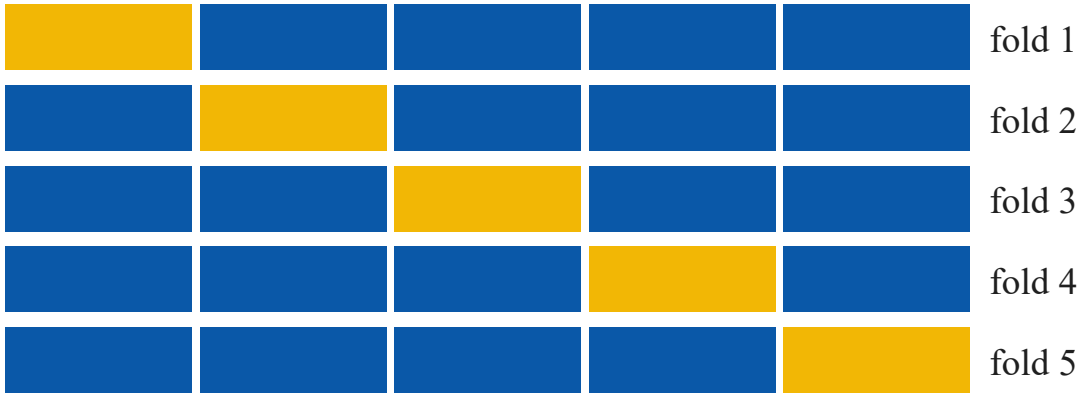
- 240 rows to learn, 60 to test
- churn rate 0.288 and 0.283: almost equal, thanks to stratify
- same random_state → same split every run

Use the test set only to score models in Step 7, never to fit, clean or tune them.

More ways to split

Step 4 · Split

k-fold cross-validation (k = 5)



- Each part is the validation part once
- Report the mean of 5 scores: steadier than one split

Time-based split



- Data with dates, predicting the future: validate on the latest months
- A random split trains on later days and checks on earlier ones, so the score looks better than it is

Group project: 5-fold for Card Churn, time-based split for Cafe Revenue.

Step 5 · Scale

What to do in this step

Usual tasks

- Pick a scaler: min-max or standard
- Fit it on the train set only
- Transform both the train and the test set
- Scale for models that use distances or weights

Know the names

- RobustScaler
- log of a skewed column

Output of this step: every column on a comparable scale, learnt from the train set.

Why scale

Step 5 · Scale

	age	income
customer A	30	15
customer B	35	35
gap	5 years	20 million
gap / std	5 / 10.5 = 0.48	20 / 8.7 = 2.30

Raw distance $\sqrt{(5^2 + 20^2)} \approx 20.6$ depends on the units: with age in months (a gap of 60), age would dominate instead.

min-max: x′ = (x − min) / (max − min)

standard: x′ = (x − mean) / std

needs scaling	does not
kNN, SVM, logistic	decision tree
MLP, k-means	Naive Bayes

After scaling, a gap is measured in the same unit for every column: standard deviations or a share of the range.

Fit on train, transform test

Step 5 · Scale


```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

cols = ["age", "income", "txn_3m"]
mm = MinMaxScaler().fit(X_train[cols])
print(pd.DataFrame(mm.transform(X_train[cols]), columns=cols).head(3).round(2))

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)    # learn mean and std on train
X_test_s = scaler.transform(X_test)           # reuse them on test
pd.DataFrame(X_train_s, columns=X.columns)[cols].head(3).round(2)
```

```
    age  income  txn_3m
0  0.58    0.79    0.93
1  0.54    0.16    0.17
2  0.17    0.04    0.07
    age  income  txn_3m
0  0.83    4.57    3.68
1  0.63   -0.22   -1.00
2 -1.11   -1.14   -1.61
```

Read the output

- min-max: values in [0, 1]
- standard: around 0; 4.57 is a very high income
- the rest of the notebook uses X_train_s, X_test_s

fit_transform on the test set puts it on a different scale from train.
Fitting on all rows before the split leaks the test set into training.

Step 6 · Model & train

What to do in this step

Usual tasks

- Choose models that match the problem type
- Build a baseline first
- Create, fit on train, predict on test
- Try a few values of the hyperparameters

Know the names

- GridSearchCV
- pipeline

Output of this step: several fitted models, ready to be scored in Step 7.

Models and their main hyperparameter

Step 6 · Model & train

classification	regression	main hyperparameter	what it controls
`LogisticRegression`	`LinearRegression`	`C (logistic)`	how closely it fits the train set
`DecisionTreeClassifier`	`DecisionTreeRegressor`	`max_depth`	how many questions in a row
`KNeighborsClassifier`	`KNeighborsRegressor`	`n_neighbors`	how many neighbours vote
`SVC`	`SVR`	`C, kernel`	fit vs margin; shape of the boundary
`GaussianNB`			
`MLPClassifier`		`hidden_layer_sizes`	neurons in each hidden layer
`KMeans (clustering)`		`n_clusters`	number of groups

Every model has fit and predict; most classifiers also have predict_proba. Hyperparameters are set by you, chosen on the train set (validation part or k-fold), never the test set. GridSearchCV tries each combination with k-fold.

Start with a baseline

Step 6 · Model & train


```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, f1_score

base = DummyClassifier(strategy="most_frequent").fit(X_train_s, y_train)
pred = base.predict(X_test_s)
print("accuracy", round(accuracy_score(y_test, pred), 3),
      "F1", round(f1_score(y_test, pred), 3))
```

```
accuracy 0.717 F1 0.0
```

The baseline

- says "no churn" for every customer
- accuracy 0.717 = share of 0s in the test set
- F1 = 0: it catches nobody

A real model must beat 0.717 accuracy and F1 0.
Against this baseline, 0.75 accuracy is only 3 points better.

Same three lines for every model

Step 6 · Model & train


```
models = {
    "Logistic": LogisticRegression(),
    "Tree": DecisionTreeClassifier(max_depth=3, random_state=0),
    "Naive Bayes": GaussianNB(),
    "kNN": KNeighborsClassifier(n_neighbors=5),
    "SVM": SVC(),
}
rows = []
for name, model in models.items():
    model.fit(X_train_s, y_train)                    # learn
    pred = model.predict(X_test_s)                   # predict 0/1
    rows.append([name, accuracy_score(y_test, pred), f1_score(y_test, pred)])
pd.DataFrame(rows, columns=["model", "accuracy", "F1"]).round(3)
```

model	accuracy	F1
baseline	0.717	0.000
Logistic	0.850	0.727
Tree	0.750	0.545
Naive Bayes	0.750	0.634
kNN	0.783	0.581
SVM	0.783	0.552

Logistic scores best on these 60 rows. With so few rows, confirm with k-fold on the train set (Section 03).

Step 7 · Evaluate

What to do in this step

Usual tasks

- Predict on the test set; put actual and predicted side by side
- Classification: confusion matrix, accuracy, F1
- Regression: MAE, MSE
- Compare with the baseline; compare train and test scores

Know the names

- precision, recall
- AUC (Section 03)
- RMSE, R^2
- RMSLE (Cafe Revenue)

Output of this step: one model chosen, with numbers that say why.

Actual vs predicted

Step 7 · Evaluate


```
logit = models["Logistic"]
pred = logit.predict(X_test_s)
proba = logit.predict_proba(X_test_s)[:, 1]
table = pd.DataFrame({"actual": y_test.values, "predicted": pred,
                      "p_churn": proba.round(2)})
table.sort_values("p_churn", ascending=False).head(8)    # highest risk first
```

	actual	predicted	p_churn
54	1	1	0.88
42	1	1	0.87
20	0	1	0.82
31	1	1	0.79
12	0	1	0.76
32	1	1	0.73
29	1	1	0.71
48	1	1	0.71

Read the table

- p_churn: probability from predict_proba
- predicted = 1 when p_churn > 0.5
- 8 riskiest customers: 6 churned (actual 1), 2 did not

Classification metrics

Step 7 · Evaluate


```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, pred)
print(pd.DataFrame(cm, index=["actual 0", "actual 1"],
                   columns=["pred 0", "pred 1"]))
tn, fp, fn, tp = cm.ravel()
print("F1 by hand:", round(2*tp / (2*tp + fp + fn), 3))
```

```
          pred 0  pred 1
actual 0      39       4
actual 1       5      12
F1 by hand: 0.727
```

	predicted 0	predicted 1
actual 0	TN **39**	FP **4**
actual 1	FN **5**	TP **12**

accuracy = (TP + TN) / all = (12 + 39) / 60 = 0.850

F1 = 2TP / (2TP + FP + FN) = 24 / 33 = 0.727

17 customers churned: the model caught 12 (TP) and missed 5 (FN).
It also flagged 4 who stayed (FP).

Regression metrics

Step 7 · Evaluate


```python
from sklearn.metrics import mean_absolute_error, mean_squared_error

actual = [20, 30, 25]
predicted = [22, 27, 25]
print("MAE", round(mean_absolute_error(actual, predicted), 2))    # (2 + 3 + 0) / 3
print("MSE", round(mean_squared_error(actual, predicted), 2))     # (4 + 9 + 0) / 3
```

```
MAE 1.67
MSE 4.33
```

day	actual	predicted	|error|	error²
1	20	22	2	4
2	30	27	3	9
3	25	25	0	0
mean			**1.67**	**4.33**

- MAE: the average size of an error, in million VND
- MSE: squares first, so one large error counts much more

Accuracy is for classification only. A regression is judged by how far the predictions are from the actual values.

Compare models, check overfitting

Step 7 · Evaluate


```
for depth in [3, None]:
    tree = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_train_s, y_train)
    print("max_depth", depth,
          "train", round(accuracy_score(y_train, tree.predict(X_train_s)), 3),
          "test", round(accuracy_score(y_test, tree.predict(X_test_s)), 3))
```

```
max_depth 3 train 0.817 test 0.75
max_depth None train 1.0 test 0.7
```

tree	train accuracy	test accuracy	reading
max_depth=3	0.817	0.750	learns rules that also hold on new customers
no limit	1.000	0.700	learns the train set by heart: below the baseline 0.717

A high train score says little about new customers. The test score does.
Train much higher than test means overfitting.

Step 8 · Deploy

What to do in this step

Usual tasks

- Pass new data through the same preprocessing
- Predict for one customer or for many
- Build an input form
- Write the predictions to a file

Know the names

- joblib: save and load a model
- streamlit
- an API

Output of this step: someone who does not code can use the model.

Predict for a new customer, with a form

Step 8 · Deploy


```
age = 30           # @param {type:"number"}
income = 12.0      # @param {type:"number"}
txn_3m = 8         # @param {type:"number"}
open_year = 2025   # @param {type:"integer"}
gender = "F"       # @param ["F", "M"]
city = "Da Nang"   # @param ["Ha Noi", "Ho Chi Minh", "Da Nang"]

row = {c: 0 for c in X.columns}
row.update({"age": age, "income": income, "txn_3m": txn_3m, "open_year": open_year,
            f"gender_{gender}": 1, f"city_{city}": 1})
p = logit.predict_proba(scaler.transform(pd.DataFrame([row])[X.columns]))[:, 1][0]
print(f"Churn risk: {p:.2f} ->", "call this customer" if p >= 0.5 else "no call")
```

```
Churn risk: 0.94 -> call this customer
```

form (what Colab shows)

age	30
income	12.0
txn_3m	8
open_year	2025
gender	F ▾
city	Da Nang ▾

Same path as training: same columns, same order, same scaler (transform, not fit).
@param turns a variable into a form field on Colab; elsewhere it is an ordinary comment.

Many customers: write a file

Step 8 · Deploy

1 2 3 4 5 6 7 8

```
1 out = pd.DataFrame({"row": X_test.index, "p_churn": proba.round(3)})
2 out.to_csv("predictions.csv", index=False)
3 out.head(3)
```

```
   row  p_churn
0  191    0.022
1  258    0.290
2  156    0.033
```

A Kaggle submission

- is a file like this one: a key column (here the row number) and a prediction

other ways to deploy	runs on	use it for
`joblib.dump / joblib.load`	anywhere	save a trained model, load it later without training again
`streamlit`	own computer (Colab: needs a tunnel)	a small web page with inputs and buttons
`an API`	a server	other applications send data and get predictions

03

Application: Card Churn

The same eight steps on the Kaggle data of the group project, ending with a first submission.

Card Churn: the problem

Competition 1 of the group project

question	which customers will cancel their credit card in the next 3 months?
train.csv	12,120 rows with the label churn_3m (0 / 1)
test.csv	8,000 rows without the label: you predict them
columns	24: customer, card, transactions, contact with the bank
metric	AUC on Kaggle: 30% of the test rows on the public board, 70% on the private board

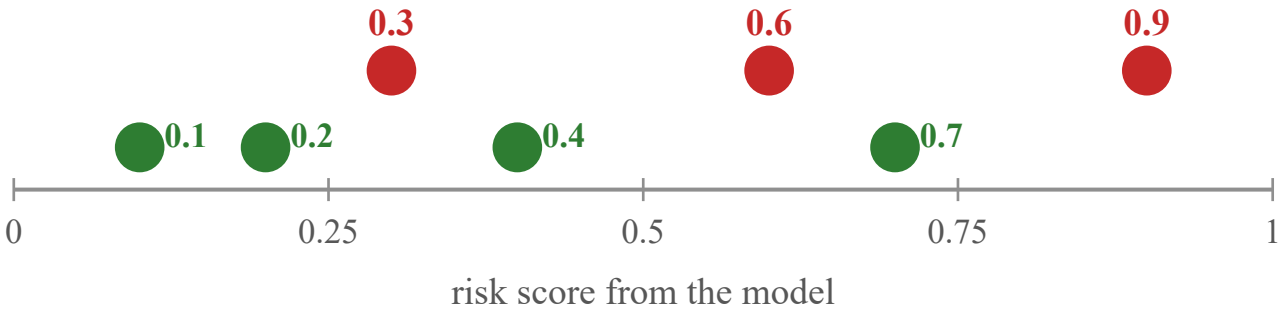
Three sets of rows in this section

train part: 80% of train.csv · learn	validation part: 20% · score, choose	test.csv · Kaggle scores it

In the code: X_train / X_val come from train.csv; X_new is test.csv.

What AUC measures

Card Churn is scored by AUC, not accuracy



red: churned (3) green: stayed (4)

churner	stayers below it	pairs won
0.9	0.7, 0.4, 0.2, 0.1	4
0.6	0.4, 0.2, 0.1	3
0.3	0.2, 0.1	2
	all pairs: 3 × 4 = 12	**9**

AUC = 9 / 12 = 0.75

AUC

- Pick a random churner and a random stayer: AUC is the chance the churner gets the higher score
- 0.5 = guessing · 1 = every pair right
- Uses the scores, not 0/1: no threshold, and unequal classes do not fool it

Read the data, first look

Card Churn · Step 1–2


```
train = pd.read_csv(f"{DATA}/train.csv")
test = pd.read_csv(f"{DATA}/test.csv")
print(train.shape, test.shape)
```

```
(12120, 24) (8000, 23)
```

```
print("churn rate:", round(train.churn_3m.mean(), 3))
train.isna().mean().sort_values(ascending=False).head(8).round(3)
```

```
churn rate: 0.117
other_bank_cards        0.203
income_month            0.129
app_logins_30d          0.125
salary_via_bank         0.059
occupation              0.051
last_contact_channel    0.042
age                     0.033
gender                  0.019
```

```
train[["age", "txn_count_3m", "spend_change_pct"]].describe().round(1)
```

	age	txn_count_3m	spend_change_pct
count	11724.0	12120.0	12120.0
mean	39.7	18.9	-0.1
std	29.9	10.3	24.9
min	0.0	1.0	-95.0
25%	31.0	12.0	-16.9
50%	38.0	17.0	-0.0
75%	45.0	24.0	16.6
max	999.0	101.0	105.8

Look for

- 11.7% churn: "nobody churns" is right 88.3% of the time
- the 8 columns with the most missing values; the highest is 20.3%
- age 0 and 999 again: the demo fixes age

Categories

Card Churn · Step 2


```
print(train.gender.value_counts(dropna=False).to_dict())
print(train.province.value_counts().head(10).to_dict())
```

gender	count
F	5,572
M	4,778
female	280
male	278
"F "	264
Nữ	255
Nam	235
(empty)	231
m	227

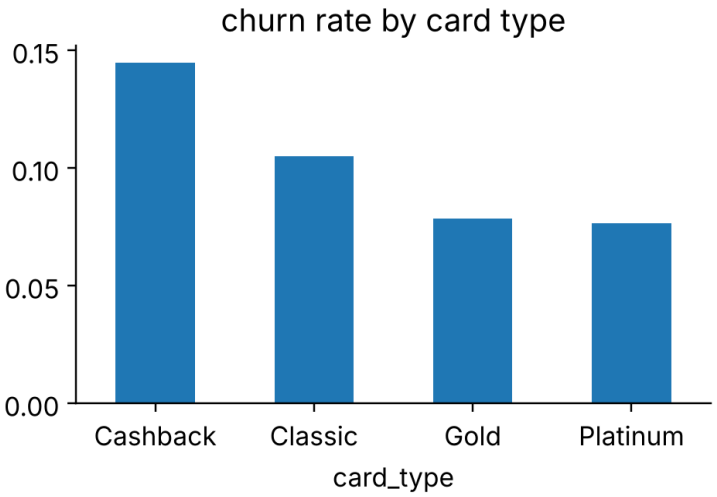
province (top 10)	count
TP.HCM	2,598
Hà Nội	2,587
Hải Phòng	639
Đà Nẵng	506
Bình Dương	424
Cần Thơ	408
Đồng Nai	293
TP HCM	247
Ho Chi Minh	244
ha noi	239

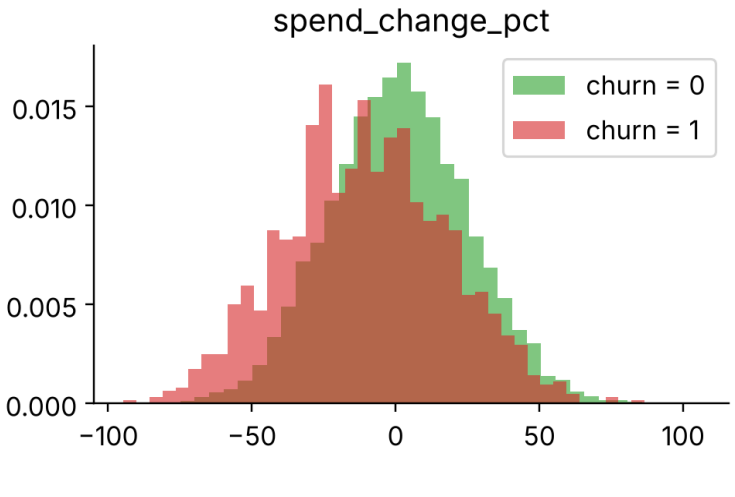
Same problem, bigger

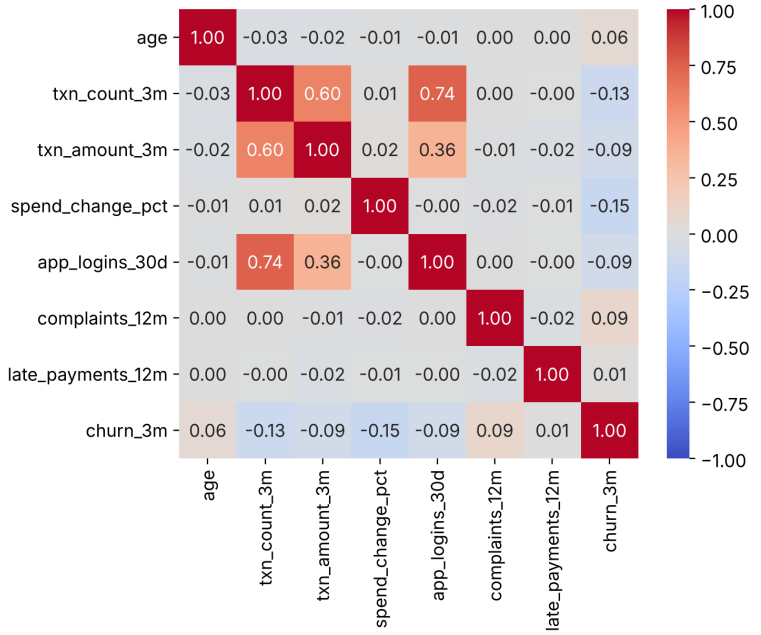
- gender: 9 values, two genders written several ways, plus missing
- province: one city written several ways
- The demo fixes gender only

Charts that matter

Card Churn · Step 2







Read the charts

- Cashback cards churn most (about 14%), Gold and Platinum least (about 8%)
- Churners (red) cut their spending more: the red bars sit further left

Row churn_3m: every correlation is small, $|r| \leq 0.15$. No single column decides.

Basic cleaning

Card Churn · Step 3


```
def clean(d):
    d = d.drop(columns=["bank_code", "currency"])                    # constant columns
    d.loc[~d.age.between(18, 100), "age"] = np.nan                  # impossible ages
    g = d.gender.str.strip().str.lower()
    d["gender"] = g.map({"m": "M", "male": "M", "nam": "M",
                         "f": "F", "female": "F", "nữ": "F"})
    return d

tr = clean(train.drop_duplicates())     # duplicates: train only
te = clean(test)                        # test keeps every row
print(train.shape, "->", tr.shape)
print(tr.gender.value_counts(dropna=False).to_dict())
```

```
(12120, 24) -> (12000, 22)
{'F': 6310, 'M': 5461, nan: 229}
```

the demo fixes	how
duplicate rows	train only
bank_code, currency	drop: constant
age 0, 999	→ NaN
gender spellings	→ M, F

Left for your group:
the other columns.
Finding the problems is part of the grade.

Features: numbers and one-hot columns

Card Churn · Step 3


```
cats = ["gender", "card_type", "salary_via_bank", "last_contact_channel"]

def features(d):
    X = d.select_dtypes("number").drop(columns="churn_3m", errors="ignore")
    return X.join(pd.get_dummies(d[cats], dtype=int))

X, y = features(tr), tr.churn_3m
X_new = features(te).reindex(columns=X.columns, fill_value=0)
print(X.shape, X_new.shape)
```

```
(12000, 25) (8000, 25)
```

Read the code

- all numeric columns, without the target
- one-hot for 4 clean text columns
- reindex: the test set gets exactly the train columns, in the same order; a missing one is added as 0

Same function for train and test: both get the same columns. 25 features.

Split, then scale inside a pipeline

Card Churn · Step 4–5


```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

def pipe(model):
    return make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), model)
```

Why a pipeline

- fit learns all three parts from the train part only
- predict reuses them: no leak, nothing to forget

`SimpleImputer`

fills NaN with the train median

→

`StandardScaler`

train mean and std

→

`model`

fit / predict

X_val plays the role that X_test played in Section 02: labelled rows kept aside to score models.

Six models and a baseline

Card Churn · Step 6–7


```
models = {
    "Baseline": DummyClassifier(strategy="most_frequent"),
    "Logistic": LogisticRegression(max_iter=2000),
    "Tree": DecisionTreeClassifier(max_depth=5, random_state=0),
    "Naive Bayes": GaussianNB(),
    "kNN": KNeighborsClassifier(n_neighbors=51),
    "SVM": SVC(),
    "MLP": MLPClassifier((16,), max_iter=500, random_state=0),
}
rows = []
for name, model in models.items():
    m = pipe(model).fit(X_train, y_train)
    pred = m.predict(X_val)
    # SVC has no predict_proba by default; its decision_function also ranks customers
    score = (m.decision_function(X_val) if name == "SVM"
             else m.predict_proba(X_val)[:, 1])
    rows.append([name, accuracy_score(y_val, pred), f1_score(y_val, pred),
                 roc_auc_score(y_val, score)])
```

model	accuracy	F1	AUC
Baseline	0.883	0.000	0.500
Logistic	0.890	0.125	0.766
Tree	0.885	0.238	0.735
Naive Bayes	0.808	0.393	0.740
kNN	0.883	0.000	0.745
SVM	0.887	0.075	0.719
MLP	0.888	0.316	0.769

Accuracy 0.81–0.89; baseline 0.883.
AUC separates: MLP 0.769, logistic 0.766.
Keep logistic: almost the same AUC, simpler.

The confusion matrix at 0.5

Card Churn · Step 7


```
1 from sklearn.metrics import confusion_matrix
2
3 best = pipe(LogisticRegression(max_iter=2000)).fit(X_train, y_train)
4 pred = best.predict(X_val)
5 cm = confusion_matrix(y_val, pred)
6 pd.DataFrame(cm, index=["actual 0", "actual 1"], columns=["pred 0", "pred 1"])
```

```
          pred 0  pred 1
actual 0    2116       3
actual 1     262      19
```

	pred 0	pred 1
actual 0	TN 2,116	FP 3
actual 1	FN 262	TP 19

At the threshold 0.5 the model flags 22 customers and catches 19 of 281 churners.
The bank calls customers in order of risk, from the top, as far as its budget goes:
AUC scores that order.

Check the score with 5-fold

Card Churn · Step 7


```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(5, shuffle=True, random_state=0)
auc = cross_val_score(pipe(LogisticRegression(max_iter=2000)), X, y,
                      cv=cv, scoring="roc_auc")
print("5-fold AUC:", auc.round(3), "mean", round(auc.mean(), 3))
```

```
5-fold AUC: [0.709 0.763 0.758 0.741 0.75 ] mean 0.744
```

Read the output

- one split: 0.766
- 5 folds: 0.709 to 0.763, mean 0.744
- one split can be lucky; the mean is steadier

Use the 5-fold mean to compare models and cleaning steps in the group project.
Kaggle public score for this model: 0.763. The test rows happen to be a little easier.

First Kaggle submission

Card Churn · Step 8


```
final = pipe(LogisticRegression(max_iter=2000)).fit(X, y)
sub = pd.DataFrame({"customer_id": te.customer_id,
                    "churn_3m": final.predict_proba(X_new)[:, 1]})
sub.to_csv("submission.csv", index=False)
print(sub.shape)
sub.head(3)
```

```
(8000, 2)
  customer_id  churn_3m
0  CUSJCOH081  0.028508
1  CUSFDBKK99  0.120951
2  CUS7DPD79Y  0.054511
```

The file

- 8,000 rows, one per test customer; public score about 0.76
- churn_3m is a probability, not 0/1: AUC needs scores

1 Join

class invite link; accept the rules

2 Open

class demo code notebook → Copy & Edit

3 Add data

Add Input → Competitions → Card Churn

4 Run

Run All; submission.csv in /kaggle/working

5 Submit

Save Version, submit the output; or Submit Prediction

The call list, and how many churners it catches

Card Churn · Step 8


```
n = int(len(sub) * 0.10)
call = sub.sort_values("churn_3m", ascending=False).head(n)
print(n, "customers to call; lowest risk on the list:", round(call.churn_3m.min(), 3))
call.head()
```

```
800 customers to call; lowest risk on the list: 0.262
```

```
from sklearn.model_selection import cross_val_predict

oof = cross_val_predict(pipe(LogisticRegression(max_iter=2000)), X, y,
                        cv=cv, method="predict_proba")[:, 1]
top = oof >= np.quantile(oof, 0.90)          # the riskiest 10% of labelled customers
print("customers:", top.sum(), "| churn rate in the list:", round(y[top].mean(), 3))
print("share of all churners in the list:", round(y[top].sum() / y.sum(), 3))
```

```
customers: 1200 | churn rate in the list: 0.388
share of all churners in the list: 0.331
```

Read the output

- call the 800 riskiest test customers (10%)
- on labelled rows, the riskiest 10%: 1,200 customers, 38.8% churn (11.7% overall)
- 33.1% of all churners are on the list

The test labels are hidden, so the estimate uses out-of-fold predictions: each labelled customer is scored by a model trained on the other 4 folds. The group report asks for this with your own model.

A form for one customer

Card Churn · Step 8


```
customer_id = "CUSV0ZEUXZ"   # @param {type:"string"}
txn_count_3m = 5              # @param {type:"integer"}
complaints_12m = 1            # @param {type:"integer"}

row = X_new[te.customer_id.values == customer_id].copy()
print("now:", int(row.txn_count_3m.iloc[0]), "transactions,",
      int(row.complaints_12m.iloc[0]), "complaints")
before = final.predict_proba(row)[:, 1][0]
row["txn_count_3m"] = txn_count_3m
row["complaints_12m"] = complaints_12m
after = final.predict_proba(row)[:, 1][0]
print(f"{customer_id}: risk {before:.2f} -> {after:.2f}")
```

form

customer_id	CUSV0ZEUXZ
txn_count_3m	5
complaints_12m	1

```
now: 23 transactions, 0 complaints
CUSV0ZEUXZ: risk 0.20 -> 0.47
```

Now: 23 transactions, no complaint, risk 0.20.
With 5 transactions and 1 complaint: risk 0.47.
A bank officer can ask "what if" without touching the code.

Practice 3

Kaggle, Card Churn · 20 min

Steps

- Follow the 5 Kaggle steps; submit submission.csv
- Change ONE thing: a model, or one more cleaning step
- Compare the 5-fold AUC before and after
- Submit again if it went up

Think

- Did the public score move the same way as your 5-fold AUC?
- Which model would you try next, and why?

Hand in Your public score, the one thing you changed, and the 5-fold AUC before and after

04

Summary and group project

Four rules to keep, what changes for Cafe Revenue, and the group project.



Summary

Eight steps, four rules

1	2	3	4	5	6	7	8
Collect & prepare	Explore	Preprocess	Split	Scale	Model & train	Evaluate	Deploy

1. Keep the test set aside

until Step 7; every decision uses the train set

2. Fit on train only

imputer and scaler: fit on train, transform test; a pipeline does it for you

3. Always compare with a baseline

"nobody churns" was right 71.9% and 88.3% of the time

4. Pick the metric for the problem

unequal classes: accuracy is not enough; F1 or AUC

Thank you

Questions? · quocthai@neu.edu.vn · fit.neu.edu.vn